DAY 1

WEEK 2 — WEB EXPLOITATION

Web Fundamentals

What is the Web?

The web is built on a client-server model. When you type a URL in your browser, your browser (the client) sends a request to a server, which responds with data — usually HTML, CSS, and JavaScript.

As a web hacker, understanding this cycle is your most fundamental skill. Every attack you'll ever do on a web application starts by understanding how data flows between the client and the server.

Key Concepts

HTTP & HTTPS

HTTP (HyperText Transfer Protocol) is the language browsers and servers use to talk to each other. HTTPS is the same but with encryption (TLS/SSL).

HTTP	Sends data in plain text — can be intercepted!
HTTPS	Encrypts data — much safer for sensitive info like passwords.

HTTP Methods

These are the 'verbs' of HTTP — they describe what action the client wants to perform:

Method	Purpose	Example
`GET`	Retrieve data from server	Load a webpage, fetch an image
`POST`	Send data to server	Submit a login form
`PUT`	Update a resource	Edit your profile info
`DELETE`	Remove a resource	Delete a post

Anatomy of a URL

```
https://www.example.com:443/path/page?id=5&name=test#section
|       |               |   |         |              |
scheme  host            port path      query params  fragment
```

HTTP Request & Response

Every time you visit a webpage, here's what happens behind the scenes:

```
── REQUEST (Browser → Server) ──────────────────────
GET /login HTTP/1.1
Host: example.com
User-Agent: Mozilla/5.0
Cookie: session=abc123

── RESPONSE (Server → Browser) ─────────────────────
HTTP/1.1 200 OK
Content-Type: text/html
Set-Cookie: session=abc123; HttpOnly

<html>...</html>
```

Common Status Codes

- 200 OK — Request succeeded
- 301/302 — Redirect to another page
- 403 Forbidden — You're not allowed here (but it exists!)
- 404 Not Found — Page doesn't exist
- 500 Internal Server Error — Server crashed (great sign for hackers!)

🛠 Tools You'll Use Today

Browser DevTools	Press F12 in any browser → Network tab to see all HTTP requests/responses live.

curl	Command-line tool to make HTTP requests manually. Pre-installed on Linux/Mac.

```
# Make a GET request
curl https://example.com

# Show response headers
curl -I https://example.com

# Send a POST request
curl -X POST -d 'username=admin&password=1234'
https://example.com/login

# Follow redirects
curl -L https://example.com
```

🎯 Your Task for Today

🚩 **Flag Challenge — Analyze a Webpage**

1. 1. Open your browser and go to: https://httpbin.org
2. 2. Open DevTools (F12) → go to the Network tab
3. 3. Visit https://httpbin.org/get — observe the request/response
4. 4. Using curl in your terminal, run:
5. curl -I https://httpbin.org/get
6. 5. Find and note: the Server header value and the Content-Type
7. 6. Now send a POST request using curl to https://httpbin.org/post
8. with data: name=YourName&club=HackShield
9. 7. Capture the JSON response — the 'form' field is your flag!
10. BONUS: Try adding a custom header: curl -H 'X-HackShield: true' https://httpbin.org/headers

📚 Reference Links

- MDN — How the Web works (Beginner)
- HTTP Status Codes — Every code explained
- curl tutorial for beginners
- PortSwigger Web Academy — Web Fundamentals
- PicoCTF Web Exploitation Challenges